

PAULT DOCUMENTATION

2026 02 19 Ai Intelligence Design

Source: docs/plans/2026-02-19-ai-intelligence-design.md

Theme: Clean Technical

Generated: February 26, 2026

AI Intelligence Depth — Design Document

Date: 2026-02-19

Status: Implemented

Plan file: .claude/plans/async-honking-dolphin.md

Goal

Add three Pro-tier AI intelligence features to Pault:

1. **Run History** — persistent record of every LLM run, viewable in the Inspector
2. **A/B Testing** — run two prompt variants concurrently, compare side-by-side, promote the winner
3. **Auto-Improvement Loop** — iterative AI-driven prompt refinement with diff view and star ratings

All features gate on `ProStatusManager.shared.isProUnlocked` (StoreKit 2).

Architecture

Data Backbone: `PromptRun`

A single `@Model` class serves all three features. The `variantLabel: String?` field discriminates run types:

| `variantLabel` | Feature |

|---|---|

| `nil` | Plain run (`ResponsePanel`) |

| "A" / "B" | A/B test run |

| "refine-1" ... "refine-5" | Refinement loop iteration |

`PromptRun` has a nullable `prompt: Prompt?` relationship — nullable so runs survive prompt deletion. `promptTitle` is a snapshot at run time for display even after deletion.

Feature Components

```
PromptDetailView
■■■ RichTextEditor (switches content/variantB binding via showVariantB)
■■■ TemplateVariablesView
■■■ AttachmentsStripView
■■■ AIAssistPanel (showAIPanel) ← NEW Task 6
■ ■■■ Tab: Improve (one-shot improve + accept/reject)
■ ■■■ Tab: Refine → RefinementLoopView ← NEW Task 6
■ ■■■ Tab: Variables (stub)
■ ■■■ Tab: Tags (stub)
■ ■■■ Tab: Score (stub)
■■■ ResponsePanel (showResponsePanel) ← NEW Task 3
■■■ Overlay toolbar
■■■ A/B segment picker ← NEW Task 5
```

```

■■■ Run A/B button ← NEW Task 5
■■■ A/B activate (a.square) ← NEW Task 5
■■■ AI Assist (sparkles) ← NEW Task 6
■■■ Run (play.circle) ← NEW Task 3
■■■ Inspector toggle (info.circle)

InspectorView (segmented: Info | History) ← NEW Task 4
■■■ Tab: History → RunHistoryView
■■■ RunHistoryRowView (expandable, copy, save-as-prompt)

ABTestResultView (sheet, 700×480) ← NEW Task 5
■■■ Two variantColumn() panes + Promote A/B footer

```

Concurrency Model

- **ResponsePanel:** Single Task drives the `AsyncThrowingStream` loop. `@State private var runTask` holds a reference for cancellation on `onDisappear`.
- **A/B Test:** `async let outputA + async let outputB` in a single Task — true parallel execution. Both streams must complete before the result sheet is shown. Total wall-clock latency $\approx \max(\text{latA}, \text{latB})$.
- **Refinement Loop:** Sequential — each iteration awaits the previous. `AIService.shared.improve` is called; history context is injected into the system prompt to guide improvement.

Pro Gating

Every feature entry point checks `ProStatusManager.shared.isProUnlocked`. On failure it sets `showPaywall = true`, which presents

`PaywallView(featureName:featureDescription:featureIcon:)`.

Run History in `InspectorView` renders `proGateView` (with `ProBadge()`) when not Pro.

Data Flow

Plain Run

```

User taps ■ → showResponsePanel = true
ResponsePanel.onAppear → AIService.streamRun → tokens stream to @State
On complete → persistRun() → PromptRun inserted → modelContext.save()
InspectorView History tab → FetchDescriptor with #Predicate → shows new row

```

A/B Run

```

User edits variant B (showVariantB = true, prompt.variantB = ...)
User taps "Run A/B"
runABTest() → async let x2 → both streams collected
MainActor.run → two PromptRun records inserted (variantLabel "A"/"B")
showABResult = true → ABTestResultView sheet
User taps "Promote B" → prompt.content = runB.resolvedInput; prompt.variantB = nil

```

Refinement Loop

```

User types goal → taps "Refine"
AIService.improve() called with history context
DiffView shows word-level diff (CollectionDifference)
User rates 1-5 stars
"Try Again" → history appended → next iteration with accumulated context
"Accept" → all intermediate PromptRun records persisted → prompt.content updated

```

Key Design Decisions

Why variantB: String? on Prompt instead of a separate model?

Keeps the A/B concept local to a single prompt with minimal schema surface. A/B is inherently binary and transient — nil means inactive. A separate model would add join complexity for a two-state toggle.

Why resolvedInput on PromptRun?

Template variables are substituted at run time. The run record captures what was *actually sent* to the LLM, not the template, enabling exact reproduction and meaningful diff display.

Why word-level diff with CollectionDifference?

Character-level diffs are too noisy for prompt text. Sentence-level misses small but important changes. Word-level strikes the right balance and uses stdlib — no additional dependency.

Why max 5 refinement iterations?

Prevents runaway API spend. 5 iterations is sufficient for meaningful convergence; users can always start a new loop.

Files Created / Modified

File	Status	Task
------	--------	------

---	---	---
-----	-----	-----

Pault/PromptRun.swift	Created	1
-----------------------	---------	---

Pault/PaultApp.swift	Modified (added PromptRun to Schema)	1
----------------------	--------------------------------------	---

PaultTests/PromptRunTests.swift	Created	1
---------------------------------	---------	---

Pault/Prompt.swift	Modified (added variantB)	2
--------------------	---------------------------	---

Pault/ResponsePanel.swift	Created	3
---------------------------	---------	---

Pault/PromptDetailView.swift	Modified (Tasks 3, 5, 6)	3, 5, 6
------------------------------	--------------------------	---------

Pault/RunHistoryView.swift	Created	4
----------------------------	---------	---

Pault/InspectorView.swift	Modified (segmented tabs)	4
---------------------------	---------------------------	---

Pault/ABTestResultView.swift	Created	5
------------------------------	---------	---

Pault/RefinementLoopView.swift	Created	6
--------------------------------	---------	---

Pault/AIAssistPanel.swift	Created	6
---------------------------	---------	---